

Week 4 Practical

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions

```
divya@Divyasree:~$ mkdir wait_waitpid
cd wait_waitpid
divya@Divyasree:~/wait_waitpid$ nano wait_waitpid.c
divya@Divyasree:~/wait_waitpid$ gcc wait_waitpid.c -o wait_waitpid
divya@Divyasree:~/wait_waitpid$ ./wait_waitpid
Parent Process Started
Parent PID: 647

Child 1 started. PID = 648

Parent is using wait()...
Child 2 started. PID = 649
Child 3 started. PID = 650
Child 1 completed. PID = 648
wait(): Child PID 648 terminated with exit status 10

Parent is using waitpid()...
Waiting specifically for Child 3 (PID = 650)
Child 2 completed. PID = 649
Child 3 completed. PID = 650
waitpid(): Child PID 650 terminated with exit status 12
Remaining child PID 649 completed.

All child processes completed.
Parent process exiting.
divya@Divyasree:~/wait_waitpid$ |
```

Code :

```
GNU nano 7.2 wait_waitpid.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0)
        {
            printf("Child %d started. PID = %d\n", i + 1, getpid());

            sleep((i + 1) * 2);
```

```

        sleep((i + 1) * 2);

        printf("Child %d completed. PID = %d\n", i + 1, getpid());
        exit(10 + i);
    }

    // Demonstrating wait()
    printf("\nParent is using wait()...\n");

    pid_t completed = wait(&status);

    if (completed > 0)
    {
        if (WIFEXITED(status))
        {
            printf("wait(): Child PID %d terminated with exit status %d\n",
                completed, WEXITSTATUS(status));
        }
    }

    // Demonstrating waitpid()
    printf("\nParent is using waitpid()...\n");
    printf("Waiting specifically for Child 3 (PID = %d)\n", child[2]);

    pid_t result = waitpid(child[2], &status, 0);

    if (result > 0)
    {

```

```

        if (result > 0)
        {
            if (WIFEXITED(status))
            {
                printf("waitpid(): Child PID %d terminated with exit status %d\n",
                    result, WEXITSTATUS(status));
            }
        }

        // Wait for remaining children
        for (int i = 0; i < 3; i++)
        {
            if (child[i] != completed && child[i] != result)
            {
                waitpid(child[i], &status, 0);

                printf("Remaining child PID %d completed.\n", child[i]);
            }
        }

        printf("\nAll child processes completed.\n");
        printf("Parent process exiting.\n");

        return 0;
    }
}

```

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
divya@Divyasree:~$ mkdir zombie_process
divya@Divyasree:~$ cd zombie_process
divya@Divyasree:~/zombie_process$ nano zombie.c
divya@Divyasree:~/zombie_process$ gcc zombie.c -o zombie
divya@Divyasree:~/zombie_process$ ./zombie
Parent process started.
Parent PID: 833
Child PID: 834
Parent is sleeping for 30 seconds...
Child process started.
Child PID: 834
Child process exiting...
Parent process exiting.
divya@Divyasree:~/zombie_process$ |
```

Code :

```
GNU nano 7.2 zombie.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process started.\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping for 30 seconds...\n");
```

```
        printf("Parent is sleeping for 30 seconds...\n");
        sleep(30);

        printf("Parent process exiting.\n");
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute

```
divya@Divyasree:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
1 Z  1000      834       833   0  80    0  -    0 -    pts/0    00:00:00 zombie
divya@Divyasree:~$ ps -ef | grep zombie
divya      833       790   0 06:52 pts/0    00:00:00 ./zombie
divya      834       833   0 06:52 pts/0    00:00:00 [zombie] <defunct>
divya      861       840   0 06:53 pts/2    00:00:00 grep --color=auto zombie
divya@Divyasree:~$
```